

thinking_mode_api_example_tool_call

```
import os
import json
from openai import OpenAI
from datetime import datetime

# The definition of the tools
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_date",
            "description": "Get the current date",
            "parameters": { "type": "object", "properties": {} },
        }
    },
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get weather of a location, the user should supply the location and date.",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": { "type": "string", "description": "The city name" },
                    "date": { "type": "string", "description": "The date in format YYYY-mm-dd" },
                },
                "required": ["location", "date"]
            }
        }
    }
]

# The mocked version of the tool calls
```

```

def get_date_mock():
    return datetime.now().strftime("%Y-%m-%d")

def get_weather_mock(location, date):
    return "Cloudy 7~13°C"

TOOL_CALL_MAP = {
    "get_date": get_date_mock,
    "get_weather": get_weather_mock
}

def run_turn(turn, messages):
    sub_turn = 1
    while True:
        response = client.chat.completions.create(
            model='deepseek-v4-pro',
            messages=messages,
            tools=tools,
            reasoning_effort="high",
            extra_body={ "thinking": { "type": "enabled" } },
        )
        messages.append(response.choices[0].message)
        reasoning_content = response.choices[0].message.reasoning_content
        content = response.choices[0].message.content
        tool_calls = response.choices[0].message.tool_calls
        print(f"Turn {turn}.
{sub_turn}\n{reasoning_content=}\n{content=}\n{tool_calls=}")
        # If there is no tool calls, then the model should get a final
        answer and we need to stop the loop
        if tool_calls is None:
            break
        for tool in tool_calls:
            tool_function = TOOL_CALL_MAP[tool.function.name]
            tool_result =
tool_function(**json.loads(tool.function.arguments))
            print(f"tool result for {tool.function.name}:
{tool_result}\n")
            messages.append({
                "role": "tool",
                "tool_call_id": tool.id,
                "content": tool_result,
            })
            sub_turn += 1
        print()

```

```
client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),
    base_url=os.environ.get('DEEPSEEK_BASE_URL'),
)

# The user starts a question
turn = 1
messages = [{
    "role": "user",
    "content": "How's the weather in Hangzhou Tomorrow"
}]
run_turn(turn, messages)

# The user starts a new question
turn = 2
messages.append({
    "role": "user",
    "content": "How's the weather in Guangzhou Tomorrow"
})
run_turn(turn, messages)
```